

DESENVOLVIMENTO DE SISTEMAS WEB I

Projeto de Fixação: Catálogo de Livros

Exercício para fazer em casa — construam um sistema completo usando a Agenda Telefônica como modelo

MVC + Bootstrap + JDBC + PostgreSQL — do zero, por conta própria

Por que este exercício

1

Praticar sozinho o que vimos em aula

Do zero: criar o projeto, modelar os dados, construir MVC completo, sem o professor codando junto.

2

Um domínio novo, o mesmo padrão

Não é a Agenda — é um Catálogo de Livros. A estrutura é igual, mas vocês tomam as decisões.

3

Fixar o ciclo inteiro

Desde "mvn archetype:generate" até uma tela funcionando com dados reais no PostgreSQL.

4

Preparar terreno para dúvidas em aula

O que travar aqui vira pauta de perguntas na próxima aula presencial.

O que vocês vão construir

Um sistema de Catálogo de Livros com o mesmo CRUD completo da Agenda Telefônica — só que sobre uma entidade Livro.

- Tela Inicial, com botão de acesso ao catálogo
- Cadastro de um novo livro
- Exclusão de um livro (com confirmação)
- Tela de listagem (grid) com todos os livros cadastrados
- Edição de um livro existente
- Dados persistidos de verdade no PostgreSQL, via JDBC

Requisitos técnicos obrigatórios

Padrão MVC

Model, View e Controller separados — exatamente como a Agenda (Aula 3)

Bootstrap

Telas estilizadas via CDN, sem CSS escrito na mão (Aula 4)

Persistência real

PostgreSQL via JDBC — nada de List em memória (Aulas 7 e 8)

DAO isolado

Toda query SQL dentro de uma classe DAO, nunca dentro do Servlet

PreparedStatement

Sempre com "?" — nunca concatenando valores direto na query

1 Criar o projeto Maven

```
mvn archetype:generate \  
  -DgroupId=br.edu.faculdade -DartifactId=catalogo-livros \  
  -DarchetypeArtifactId=maven-archetype-webapp -DinteractiveMode=false
```

Antes de configurar o pom.xml, rodem "java -version" no terminal de vocês e anotem o resultado — no próximo passo, a versão do Java no pom precisa bater com a que está instalada na SUA máquina, não necessariamente a 25 usada em aula.

2

Configurar o pom.xml

```
<properties>
  <maven.compiler.source>SUA_VERSAO_AQUI</maven.compiler.source>
  <maven.compiler.target>SUA_VERSAO_AQUI</maven.compiler.target>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.1.0</version>
    <scope>provided</scope>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.13</version>
  </dependency>
</dependencies>
```

Troquem "SUA_VERSAO_AQUI" pelo número que apareceu no "java -version" (ex: 21, 23, 25...).

3

Estrutura de pastas esperada

```
catalogo-livros/  
  pom.xml  
  src/main/java/  
    br/edu/faculdade/  
      model/  
        Livro.java  
      controller/  
        (Servlets – Passo 6)  
      dao/  
        ConnectionFactory.java  
        LivroDAO.java  
  src/main/webapp/  
    index.jsp  
    catalogo.jsp  
    novo-livro.jsp  
    editar-livro.jsp  
  WEB-INF/  
    web.xml
```

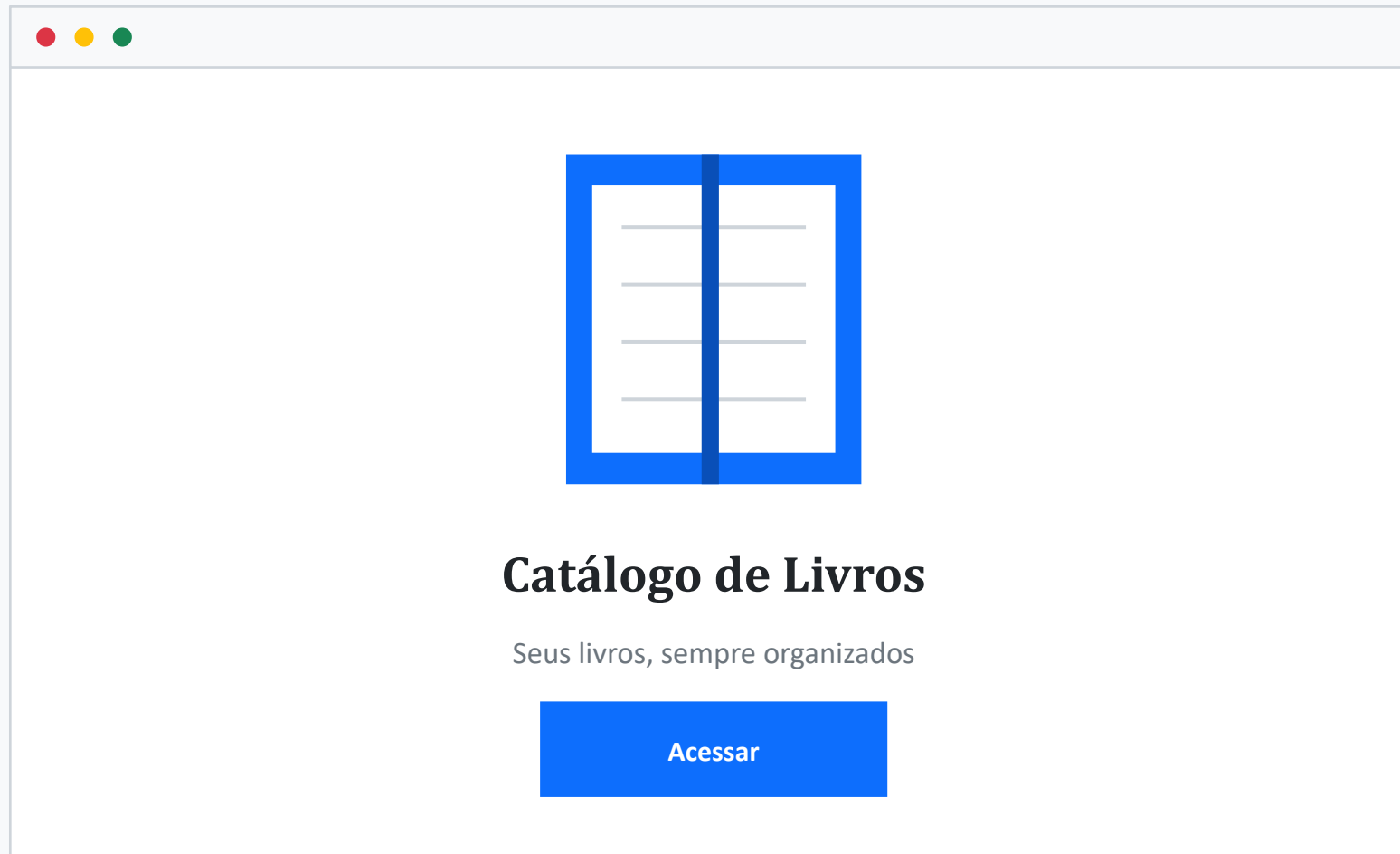
O modelo de dados: Livro

Campos que a entidade Livro precisa ter:

id	identificador único (chave primária)
titulo	título do livro
autor	nome do autor
anoPublicacao	ano de publicação (número)

Vocês decidem os tipos e as constraints (PK, NOT NULL, tamanhos de VARCHAR) e criam a tabela — no pgModeler ou direto em SQL — igual fizemos com a tabela "contato" na Aula 7. Não vamos entregar o SQL pronto desta vez.

Tela Inicial



Reproveitem a mesma estrutura da index.jsp da Agenda — só mudam texto e ícone

Catálogo de Livros

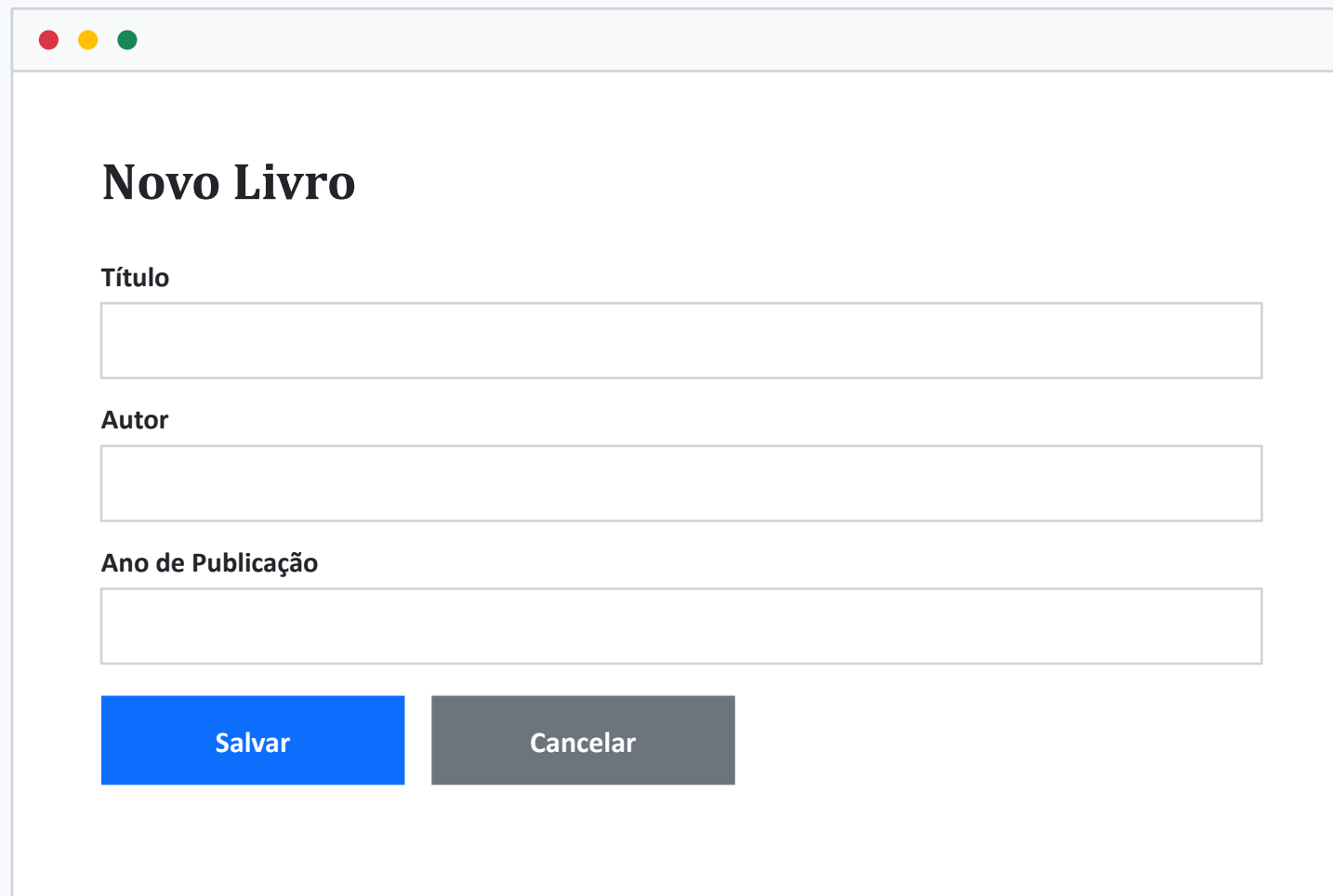
Catálogo de Livros

+ Novo Livro

ID	Título	Autor	Ano	Opções	
1	Dom Casmurro	Machado de Assis	1899	Editar	Excluir
2	O Cortiço	Aluísio Azevedo	1890	Editar	Excluir
3	Vidas Secas	Graciliano Ramos	1938	Editar	Excluir

Mesmo padrão visual da agenda.jsp — table table-striped, botões btn-warning/btn-danger

Novo Livro / Editar Livro



A screenshot of a web form titled "Novo Livro" (New Book). The form is contained within a window-like frame with three colored dots (red, yellow, green) in the top-left corner. The form itself has a title "Novo Livro" in a large, bold, black font. Below the title are three input fields, each with a label to its left: "Título" (Title), "Autor" (Author), and "Ano de Publicação" (Publication Year). Each label is in a bold, black font. The input fields are empty rectangular boxes. At the bottom of the form are two buttons: a blue button labeled "Salvar" (Save) and a gray button labeled "Cancelar" (Cancel).

Novo Livro

Título

Autor

Ano de Publicação

Salvar **Cancelar**

A tela de Editar é IGUAL, só com os campos pré-preenchidos — mesma lógica de editar-contato.jsp

Model e DAO

Livro.java

Model

id, titulo, autor, anoPublicacao + construtor + getters/setters — igual Contato.java

ConnectionFactory.java

DAO

Mesma lógica da Agenda: URL, usuário e senha do banco em um só lugar

LivroDAO.java

DAO

listar() · buscarPorId(id) · inserir(livro) · atualizar(livro) · excluir(id) — os 5 métodos de sempre, agora com SQL de "livro"

Controllers e Views

CatalogoServlet	Controller	doGet: chama LivroDAO.listar() e faz forward para catalogo.jsp
NovoLivroServlet	Controller	doGet mostra o formulário vazio · doPost lê os campos e chama LivroDAO.inserir()
EditarLivroServlet	Controller	doGet busca o livro pelo id · doPost lê os campos e chama LivroDAO.atualizar()
ExcluirLivroServlet	Controller	doPost recebe o id e chama LivroDAO.excluir() — sempre POST, nunca um link GET
4 arquivos .jsp	View	index.jsp, catalogo.jsp, novo-livro.jsp, editar-livro.jsp — Bootstrap + JSTL/EL

Checklist final



Projeto Maven completo, abrindo e rodando no Tomcat sem erros



Tabela livro criada no PostgreSQL, com PK e constraints definidas por vocês



As 4 operações do CRUD funcionando de ponta a ponta (testadas por vocês)



Nenhuma query SQL fora do LivroDAO



Todas as queries usando PreparedStatement com "?"



Telas com Bootstrap, no mesmo padrão visual da Agenda

Dicas finais

1

Consultem o código da Agenda Telefônica como referência — mas não copiem e cole 1:1. Entendam cada trecho antes de adaptar.

2

Construam e testem uma funcionalidade de cada vez: primeiro Read, depois Create, depois Update, depois Delete.

3

Erro mais comum: esquecer de trocar o nome da tabela/coluna ao adaptar SQL da Agenda para o Livro.

4

Se travar em algo específico, anotem exatamente onde — isso vira pauta da próxima aula.

BOA CONSTRUÇÃO!

Do zero até um CRUD completo — sozinhos.

Tragam dúvidas específicas para a próxima aula — vamos revisar juntos os pontos onde a turma mais travou.